1 hidden cell

```python
# TikTok Video Analysis Project
# ---------------------------------
# This notebook explores a TikTok dataset to analyze video performance and author status.
# The goal is to uncover insights about video engagement and the impact of author ban status.

# Import essential packages for data manipulation
import pandas as pd
import numpy as np

# Import packages for data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Load the TikTok dataset into a pandas DataFrame
# The dataset contains information about video metrics and author status
data = pd.read_csv("tiktok_dataset.csv")

# Display the first 10 rows to get an overview of the dataset structure and contents
data.head(10)
```

	clai...	vi...	video_duration...	video_transcription_text	verified_st...	author_ban_s...	video_view...	video_like_...	video_share_...
0	1 claim	7017666017	59	someone shared with me that drone deliveri...	not verified	under review	343296	19425	
1	2 claim	4014381136	32	someone shared with me that there are mor...	not verified	active	140877	77355	19
2	3 claim	9859838091	31	someone shared with me that american indu...	not verified	active	902185	97690	2
3	4 claim	1866847991	25	someone shared with me that the metro of s...	not verified	active	437506	239954	34
4	5 claim	7105231098	19	someone shared with me that the number of...	not verified	active	56167	34987	4
5	6 claim	8972200955	35	someone shared with me that gross domesti...	not verified	under review	336647	175546	62
6	7 claim	4958886992	16	someone shared with me that elvis presley h...	not verified	active	750345	486192	193
7	8 claim	2270982263	41	someone shared with me that the best sellin...	not verified	active	547532	1072	
8	9 claim	5235769692	50	someone shared with me that about half of t...	not verified	active	24819	10160	1
9	10 claim	4660861094	45	someone shared with me that it would take ...	verified	active	931587	171051	67

Rows: 10

Expand

```python
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19382 entries, 0 to 19381
Data columns (total 12 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   #                         19382 non-null  int64
 1   claim_status              19084 non-null  object
 2   video_id                  19382 non-null  int64
 3   video_duration_sec        19382 non-null  int64
 4   video_transcription_text  19084 non-null  object
 5   verified_status           19382 non-null  object
 6   author_ban_status         19382 non-null  object
 7   video_view_count          19084 non-null  float64
 8   video_like_count          19084 non-null  float64
 9   video_share_count         19084 non-null  float64
 10  video_download_count      19084 non-null  float64
 11  video_comment_count       19084 non-null  float64
dtypes: float64(5), int64(3), object(4)
memory usage: 1.8+ MB
```

```python
data.describe()
```

	#	video_id	video_duration...	video_view_c...	video_like_co...	video_share_...	video_download_c...	video_comment_...
count	19382	19382	19382	19084	19084	19084	19084	19084
mean	9691.5	5627454067.339129	32.4217315035	254708.5586879061	84304.6360301824	16735.2483232027	1049.4296269126	349.3121463006
std	5595.2457944223	2536440464.1693673	16.2299671785	322893.2808137871	133420.5468136723	32036.1743501506	2004.2998939792	799.6388647869
min	1	1234959018	5	20	0	0	0	0
25%	4846.25	3430416807.25	18	4942.5	810.75	115	7	1
50%	9691.5	5618663579	32	9954.5	3403.5	717	46	9
75%	14536.75	7843960211.25	47	504327	125020	18222	1156.25	292
max	19382	9999873075	60	999817	657830	256130	14994	9599

Rows: 8 Expand

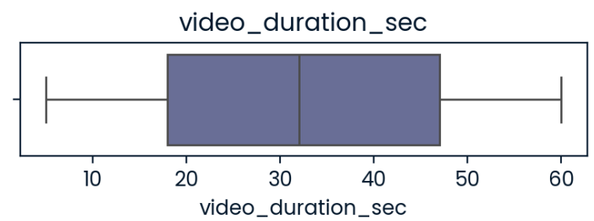
```python
data.size
```

```
232584
```

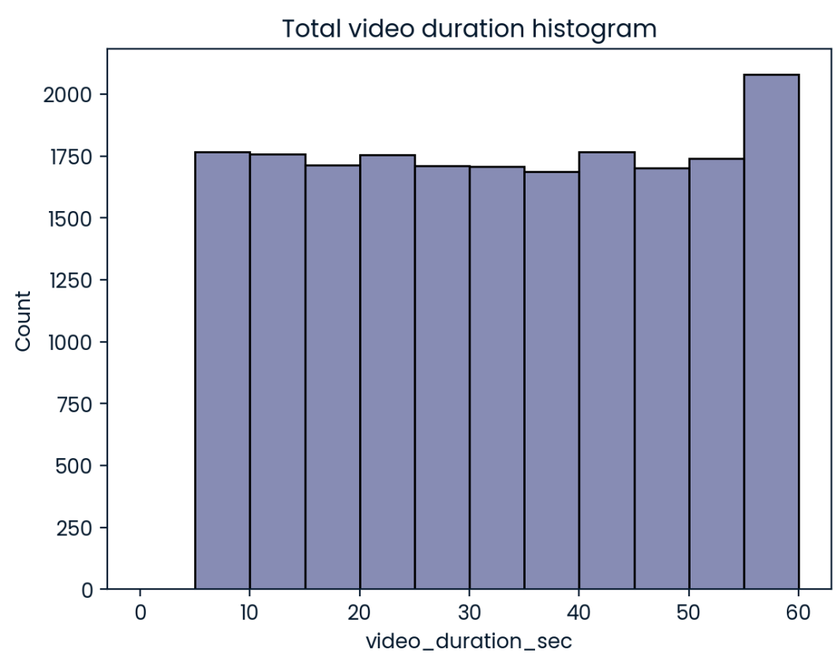
```python
# Get the shape of the data
data.shape
```

```
(19382, 12)
```

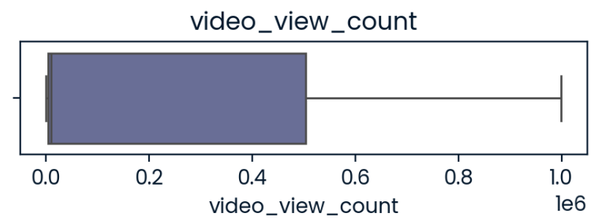
```python
# Visualize the distribution of video durations
# ---------------------------------------------
# This boxplot provides a summary of the spread and outliers in the `video_duration_sec` column.
# It helps identify typical video lengths and any unusually short or long videos in the dataset.

plt.figure(figsize=(5,1))
plt.title('video_duration_sec')
sns.boxplot(x=data['video_duration_sec']);
```



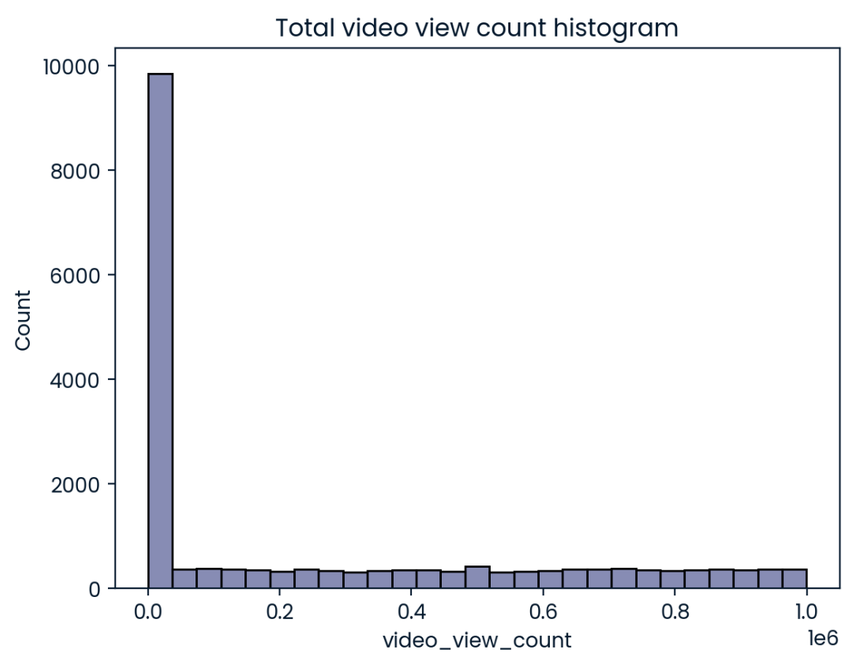

```python
# Visualize the distribution of video durations using a histogram
# ---------------------------------------------------------------
# This histogram shows the frequency of videos within specific duration intervals (bins of 5 seconds, up to 60 seconds).
# It helps to understand the most common video lengths and whether the dataset is skewed towards shorter or longer videos.
# Outliers and patterns in video duration can be easily spotted with this visualization.

sns.histplot(data['video_duration_sec'], bins=range(0, 61, 5)).set_title("Total video duration histogram")
plt.show()
```



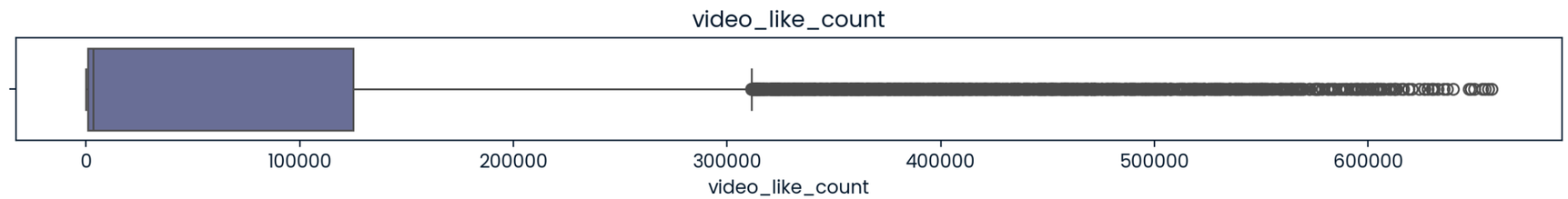
```
# Visualize the distribution of video view counts
# ----------------------------------------------
# This boxplot provides a summary of the spread and outliers in the `video_view_count` column.
# It helps identify typical view counts and any unusually low or high viewed videos in the dataset.
# This visualization is useful for spotting patterns, skewness, and potential anomalies in video popularity.

plt.figure(figsize=(5,1))
plt.title('video_view_count')
sns.boxplot(x=data['video_view_count']);
```



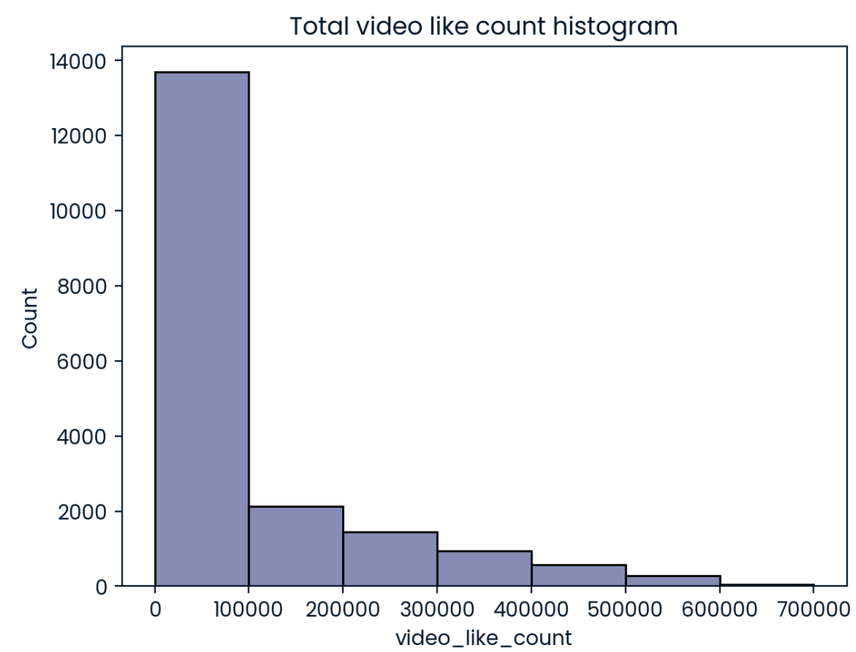

```
# Visualize the distribution of video view counts using a histogram
# ------------------------------------------------------------------
# This histogram displays how frequently different ranges of video view counts occur in the dataset.
# It helps to identify the most common view counts and whether the data is skewed towards videos with low or high views.
# Patterns, outliers, and the overall spread of video popularity can be easily observed with this plot.

sns.histplot(data['video_view_count']).set_title("Total video view count histogram")
plt.show()
```



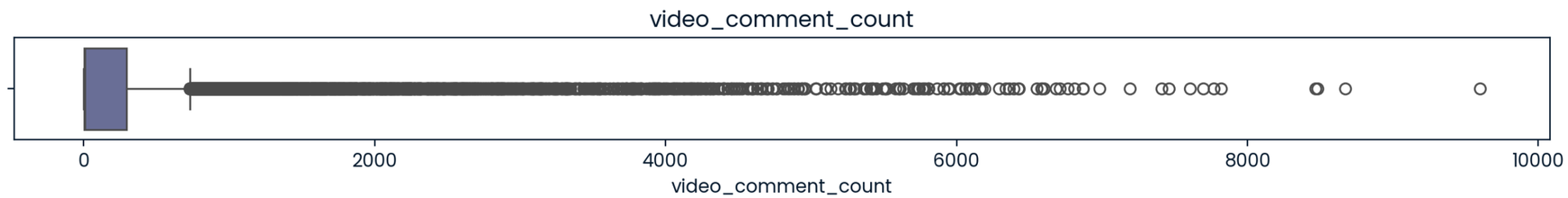
```python
# Visualize the distribution of video like counts
# ----------------------------------------------
# This boxplot provides a summary of the spread and outliers in the `video_like_count` column.
# It helps identify typical like counts and any unusually low or high liked videos in the dataset.
# This visualization is useful for spotting patterns, skewness, and potential anomalies in video popularity.

plt.figure(figsize=(15,1))
plt.title('video_like_count')
sns.boxplot(x=data['video_like_count']);
```



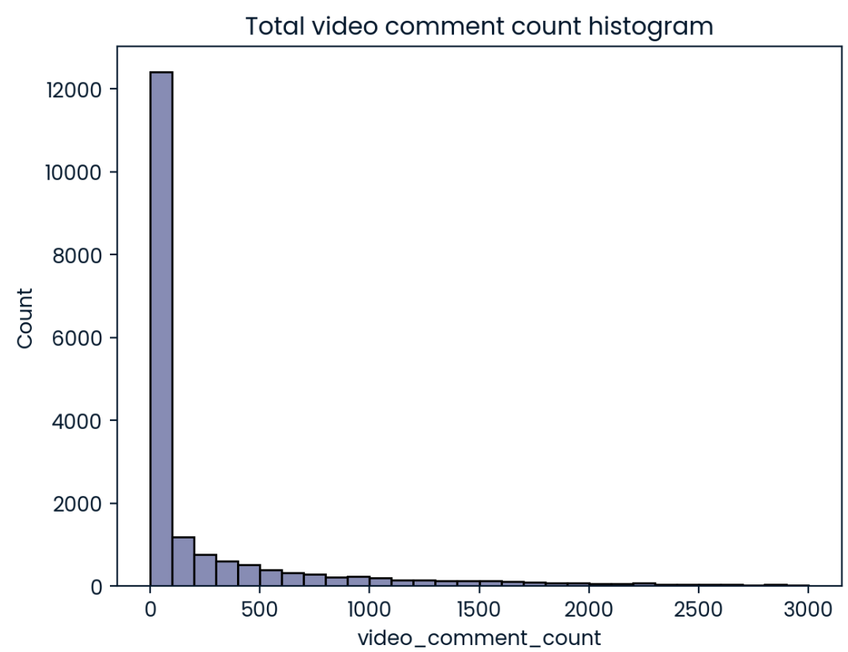
```python
# Visualize the distribution of video like counts using a histogram
# ----------------------------------------------------------------
# This histogram shows how frequently different ranges of video like counts occur in the dataset.
# By setting bins of 100,000 likes (from 0 up to 700,000), we can observe the spread and concentration of like counts.
# This helps identify whether most videos receive few likes or if there are many highly-liked videos.
# The plot is useful for detecting skewness, common like ranges, and potential outliers in video popularity.

sns.histplot(data['video_like_count'], bins=range(0, (7*10**5+1), 10**5)).set_title("Total video like count histogram")
plt.show()
```



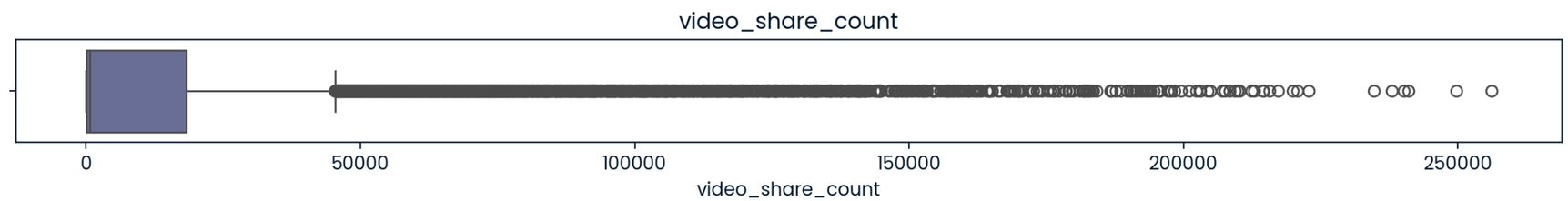
```python
# Visualize the distribution of video comment counts
# ---------------------------------------------------
# This boxplot provides a summary of the spread and outliers in the `video_comment_count` column.
# It helps identify typical comment counts and any unusually low or high commented videos in the dataset.
# This visualization is useful for spotting patterns, skewness, and potential anomalies in video engagement.

plt.figure(figsize=(15,1))
plt.title('video_comment_count')
sns.boxplot(x=data['video_comment_count']);
```



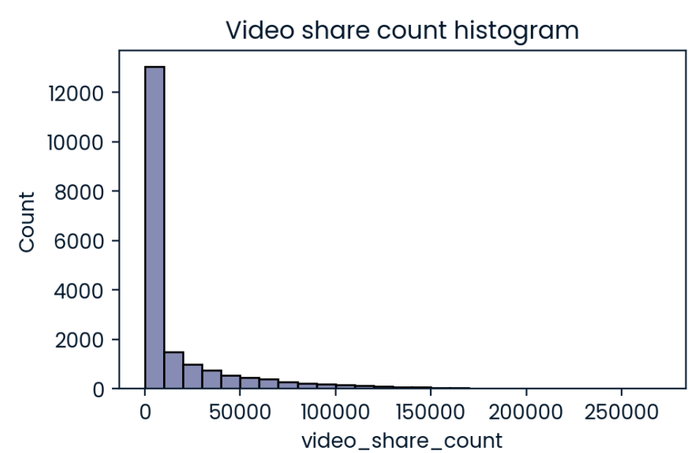
```python
# Visualize the distribution of video comment counts using a histogram
# ----------------------------------------------------------------------
# This histogram displays how frequently different ranges of video comment counts occur in the dataset.
# By setting bins of 100 comments (from 0 up to 3,000), we can observe the spread and concentration of comment counts.
# This helps identify whether most videos receive few comments or if there are many highly-commented videos.
# The plot is useful for detecting skewness, common comment ranges, and potential outliers in video engagement.

sns.histplot(data['video_comment_count'], bins=range(0, (3001), 100)).set_title("Total video comment count histogram")
plt.show()
```



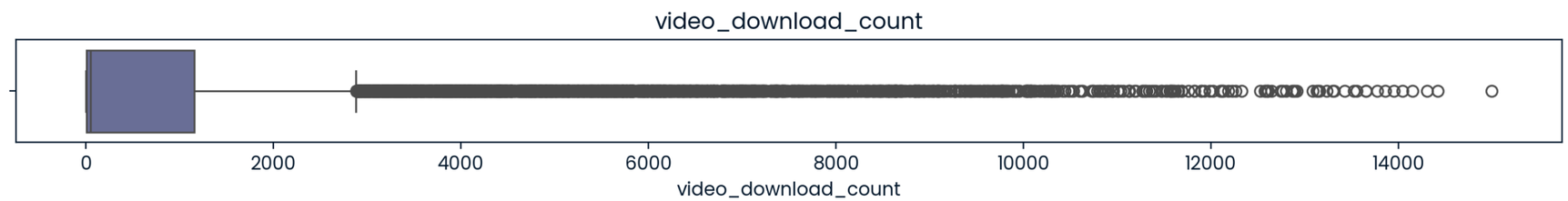
```python
# Visualize the distribution of video share counts
# ----------------------------------------------
# This boxplot provides a summary of the spread and outliers in the `video_share_count` column.
# It helps identify typical share counts and any unusually low or high shared videos in the dataset.
# This visualization is useful for spotting patterns, skewness, and potential anomalies in video sharing behavior.

plt.figure(figsize=(15,1))
plt.title('video_share_count')
sns.boxplot(x=data['video_share_count']);
```



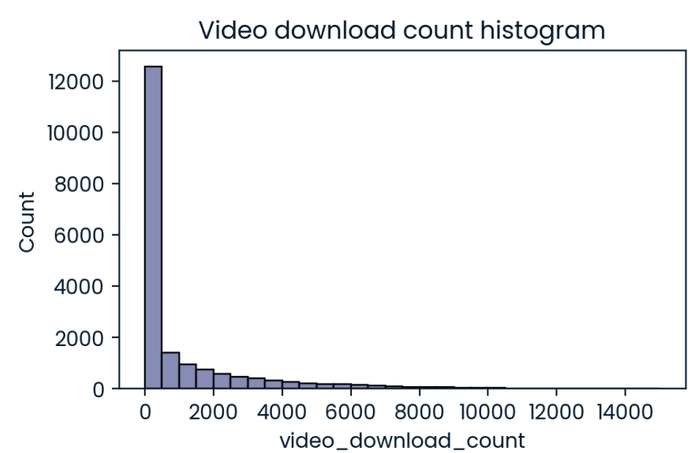

```python
# Visualize the distribution of video share counts using a histogram
# ----------------------------------------------------------------
# This histogram shows how frequently different ranges of video share counts occur in the dataset.
# By using bins of 10,000 shares (from 0 up to 270,000), we can observe the spread and concentration of share counts.
# This helps identify whether most videos receive few shares or if there are many highly-shared videos.
# The plot is useful for detecting skewness, common share ranges, and potential outliers in video engagement.

plt.figure(figsize=(5,3))
sns.histplot(data['video_share_count'], bins=range(0, (270001), 10000))
plt.title('Video share count histogram');
```



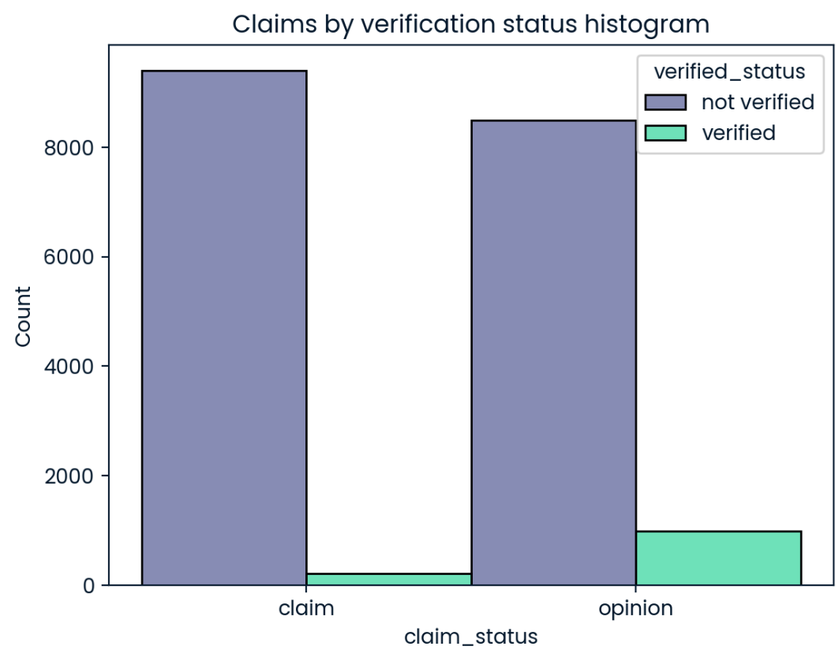
```python
# Visualize the distribution of video download counts
# ---------------------------------------------------
# This boxplot provides a summary of the spread and outliers in the `video_download_count` column.
# It helps identify typical download counts and any unusually low or high downloaded videos in the dataset.
# This visualization is useful for spotting patterns, skewness, and potential anomalies in video download behavior.

plt.figure(figsize=(15,1))
plt.title('video_download_count')
sns.boxplot(x=data['video_download_count']);
```



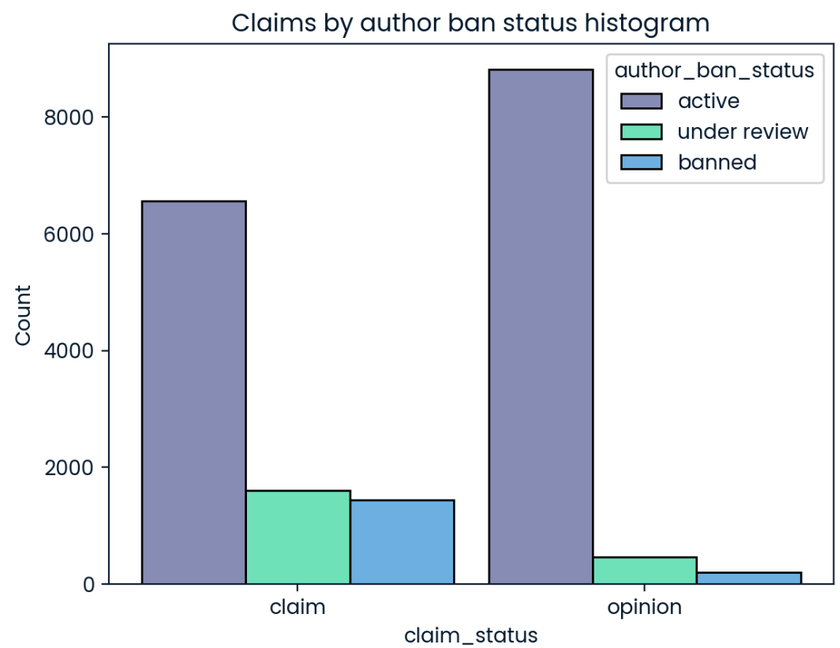

```python
# Visualize the distribution of video download counts using a histogram
# ----------------------------------------------------------------------
# This histogram displays how often different ranges of video download counts appear in the dataset.
# By using bins of 500 downloads (from 0 up to 15,000), we can see the spread and concentration of download counts.
# This helps determine whether most videos are downloaded infrequently or if there are many highly-downloaded videos.
# The plot is useful for detecting skewness, common download ranges, and potential outliers in video engagement.

plt.figure(figsize=(5,3))
sns.histplot(data['video_download_count'], bins=range(0,(15001),500))
plt.title('Video download count histogram');
```



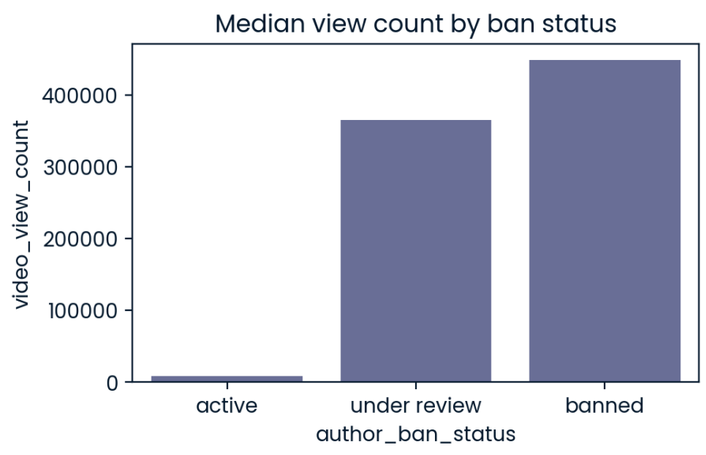
```python
# Visualize the distribution of claim statuses by verification status
# ------------------------------------------------------------------
# This histogram shows how the `claim_status` categories are distributed, separated by the `verified_status` of each video.
# The `hue='verified_status'` argument colors the bars according to whether the video is from a verified or unverified source.
# The `multiple='dodge'` option places bars for each verification status side by side for easier comparison within each claim status.
# This visualization helps identify patterns in claim status frequency and whether verification status is associated with certain claim types.

sns.histplot(
    x='claim_status',
    hue='verified_status',
    multiple='dodge',
    data=data
).set_title('Claims by verification status histogram');
```



```python
# Visualize the distribution of claim statuses by author ban status
# ----------------------------------------------------------------
# This histogram displays how the `claim_status` categories are distributed, separated by the `author_ban_status` of each video.
# The `hue='author_ban_status'` argument colors the bars according to the ban status of the video's author (e.g., active, under review, banned).
# The `hue_order=['active', 'under review', 'banned']` ensures a consistent and meaningful order of ban statuses in the plot.
# The `multiple='dodge'` option places bars for each ban status side by side within each claim status, making comparisons easier.
# The `shrink=0.9` slightly reduces bar width to prevent overlap and improve readability.
# This visualization helps identify patterns in claim status frequency and whether certain claim types are associated with specific author ban statuses.

sns.histplot(
    x='claim_status',
    hue='author_ban_status',
    multiple='dodge',
    data=data,
    hue_order=['active', 'under review', 'banned'],
    shrink=0.9
).set_title('Claims by author ban status histogram');
```



```python
# Visualize the median video view count by author ban status
# ---------------------------------------------------------
# This bar plot shows the median number of views for videos, grouped by the `author_ban_status`.
# First, the code calculates the median of all numeric columns in the `data` DataFrame for each unique `author_ban_status`.
# The resulting `median_ban_status_count` DataFrame contains the median values, including `video_view_count`, for each ban status group.
# The `sns.barplot` function then creates a bar plot with:
#   - `x='author_ban_status'`: Ban status categories on the x-axis.
#   - `y='video_view_count'`: Median view count on the y-axis.
#   - `order=['active', 'under review', 'banned']`: Ensures a consistent and meaningful order of ban statuses.
# The plot's title, 'Median view count by ban status', summarizes the visualization's purpose.
# This visualization helps identify whether certain author ban statuses are associated with higher or lower median video view counts.

median_ban_status_count = data.groupby(['author_ban_status']).median(
    numeric_only=True).reset_index()
fig = plt.figure(figsize=(5,3))
sns.barplot(
    data=median_ban_status_count,
    x='author_ban_status',
    y='video_view_count',
    order=['active', 'under review', 'banned']
).set_title('Median view count by ban status');
```



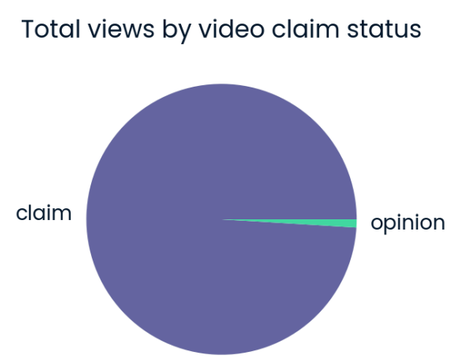
```python
# Calculate the median video view count by claim status
# -----------------------------------------------------
# This code groups the `data` DataFrame by the `claim_status` column, which represents the verification or factual status of each claim.
# For each unique `claim_status` group, it computes the median value of the `video_view_count` column.
# The result is a Series showing the median number of views for videos in each claim status category.
# This analysis helps reveal whether certain claim statuses are associated with higher or lower typical video view counts.

data.groupby('claim_status')['video_view_count'].median()
```

claim_status	video_view_count
claim	501555
opinion	4953

Rows: 2

Expand

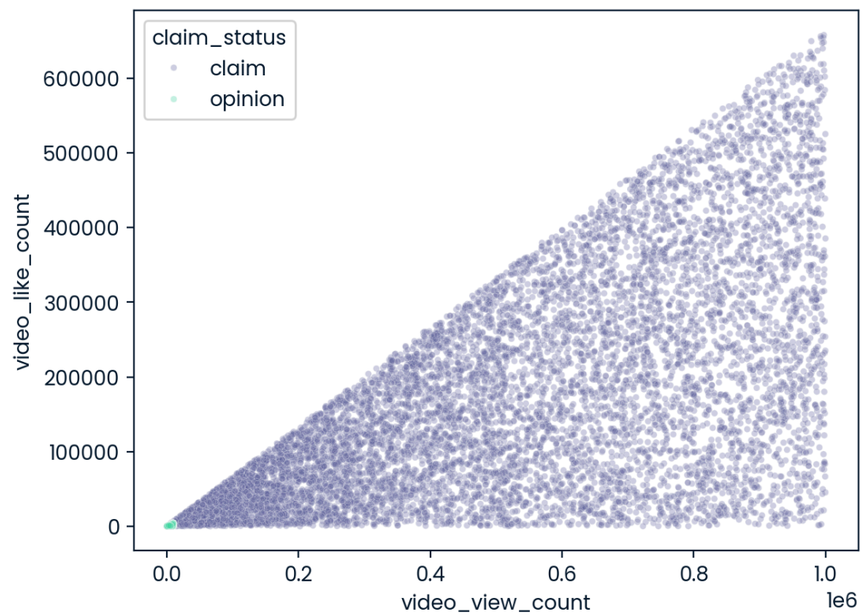
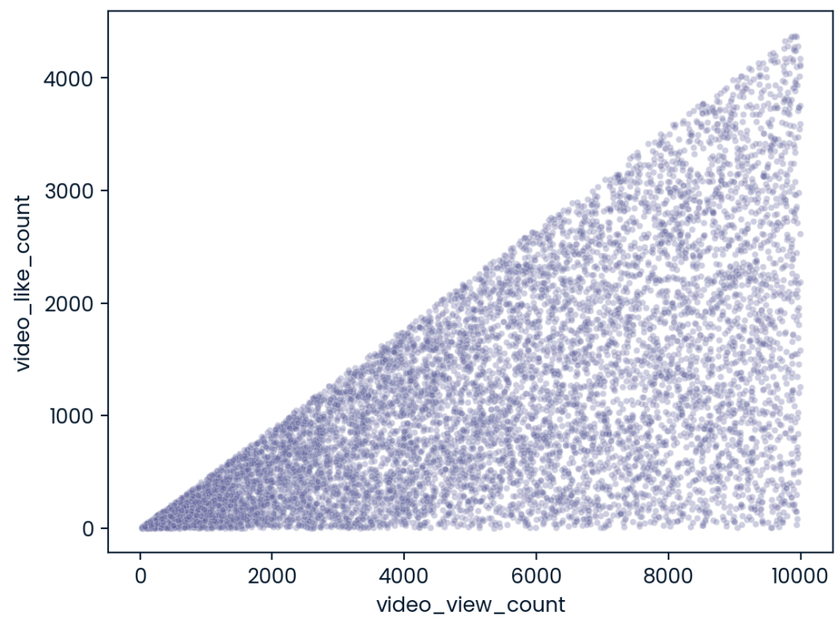
```python
# Visualize the total video views by claim status using a pie chart
# -----------------------------------------------------------------
# This code creates a pie chart to illustrate the proportion of total video views
# attributed to each claim status category in the dataset.
# First, it groups the `data` DataFrame by the `claim_status` column and sums the `video_view_count`
# for each group, resulting in the total views per claim status.
# The pie chart uses these summed values to show the relative share of views for each status.
# The chart is labeled with the claim status categories and titled for clarity.

fig = plt.figure(figsize=(3,3))
plt.pie(
    data.groupby('claim_status')['video_view_count'].sum(),
    labels=['claim', 'opinion']
)
plt.title('Total views by video claim status');
```



```python
# Identify and count outliers in video engagement metrics
# -----------------------------------------------------
# This code iterates over a list of key video engagement columns in the `data` DataFrame,
# including view, like, share, download, and comment counts.
# For each column, it calculates the first quartile (Q1), third quartile (Q3), and the interquartile range (IQR).
# The outlier threshold is defined as the median plus 1.5 times the IQR, which is a variation on the standard Tukey method.
# It then counts how many values in each column exceed this threshold, identifying potential outliers in the data.
# The number of outliers for each metric is printed, helping to assess the prevalence of unusually high engagement values.

count_cols = ['video_view_count',
              'video_like_count',
              'video_share_count',
              'video_download_count',
              'video_comment_count',
              ]

for column in count_cols:
    q1 = data[column].quantile(0.25)
    q3 = data[column].quantile(0.75)
    iqr = q3 - q1
    median = data[column].median()
    outlier_threshold = median + 1.5*iqr

    # Count the number of values that exceed the outlier threshold
    outlier_count = (data[column] > outlier_threshold).sum()
    print(f'Number of outliers, {column}:', outlier_count)
```

```
Number of outliers, video_view_count: 2343
Number of outliers, video_like_count: 3468
Number of outliers, video_share_count: 3732
Number of outliers, video_download_count: 3733
Number of outliers, video_comment_count: 3882
```

```python
# Visualize the relationship between video views and likes by claim status
# ----------------------------------------------------------------------
# This scatterplot helps to explore how the number of views relates to the number of likes
# for each video in the dataset, with points colored by the 'claim_status' category.
# Each point represents a single video, with its x-coordinate as the view count and y-coordinate as the like count.
# The 'hue' parameter distinguishes videos by their claim status, allowing for comparison across categories.
# Point size is set small (s=10) and transparency is increased (alpha=0.3) to reduce overplotting in dense regions.
# This visualization can reveal patterns or differences in engagement metrics based on claim status.

sns.scatterplot(
    x=data["video_view_count"],
    y=data["video_like_count"],
    hue=data["claim_status"],
    s=10,
    alpha=0.3
)
plt.show()
```



```python
# Visualize the relationship between video views and likes for opinion videos
# ---------------------------------------------------------------------------
# This scatterplot focuses exclusively on videos labeled as 'opinion' in the dataset.
# By filtering the data to include only rows where 'claim_status' is 'opinion',
# we can specifically analyze engagement patterns for this category.
# Each point represents a single opinion video, with the x-axis showing the view count
# and the y-axis showing the like count.
# The point size is kept small (s=10) and transparency is increased (alpha=0.3)
# to minimize overplotting and better reveal dense clusters.
# This visualization helps to identify trends or outliers in how opinion videos perform
# in terms of views and likes.

opinion = data[data['claim_status'] == 'opinion']
sns.scatterplot(
    x=opinion["video_view_count"],
    y=opinion["video_like_count"],
    s=10,
    alpha=0.3
)
plt.show()
```